Ben-Gurion University of the Negev
The Faculty of Natural Sciences
The Department of **Computer Science**

Computing High Moments of the Number of Missing 2-Grams in a Long Random Text

Or Kiraly

A thesis submitted in partial fulfillment of the requirements
for the Master of Sciences degree

Advisors:
Prof. Gera Weiss,
and **Prof. Oded Margalit**

January 2026

Ben-Gurion University of the Negev
The Faculty of Natural Sciences
The Department of **Computer Science**

Computing High Moments of the Number of Missing 2-Grams in a Long Random Text

Or Kiraly

A thesis submitted in partial fulfillment of the requirements
for the Master of Sciences degree

Under the supervision of
Prof. Gera Weiss & Prof. Oded Margalit

Signature of student: _____ Date: February 1, 2026
Signature of advisor: _____ Date: February 1, 2026
Signature of advisor: _____ Date: February 1, 2026
Signature of chairperson of the
committee for graduate studies: _____ Date: February 1, 2026

February 2026

Abstract

We study the distribution of the number of missing 2-grams in a random sequence over a finite alphabet, a quantity that plays a central role in the analysis and calibration of randomness tests. While existing approaches based on generating functions can compute only the expectation and variance of this random variable, no general method is known for exact computation of higher-order moments. We present an algorithmic method for computing arbitrary high-order moments exactly. Our approach models pattern avoidance as a path-counting problem on a deterministic finite automaton, whose transition structure captures the overlap constraints between forbidden patterns. By exploiting graph symmetries to group combinatorially equivalent states and applying the Cayley–Hamilton theorem to reduce matrix powers to low-degree polynomial recurrences, we obtain an exact method that uses only integer arithmetic and scales to the large parameter regimes encountered in practice. Beyond resolving a long-standing limitation of generating-function-based techniques, our results enable fine-grained analysis of the tail behavior of omission distributions, with potential applications in randomness testing and related combinatorial counting problems.

Acknowledgments

I would like to express my gratitude to my advisors, Prof. Gera Weiss and Prof. Oded Margalit, for their guidance, insight, and continuous support throughout this research. Their deep knowledge and encouragement were instrumental in shaping both the direction and the rigor of this work.

I would also like to acknowledge Yahav Azran for his earlier work on the automata and matrix based analysis of the OPSO test, conducted under the guidance of Prof. Gera Weiss and Prof. Oded Margalit.

Contents

Acknowledgments **ii**

1 Introduction **1**

2 Preliminaries and Background **3**

2.1 Probability Background 3

2.1.1 Standardized Moments: Skewness and Kurtosis . . . 3

2.1.2 Linearity of Expectation and Indicator Variables . . . 5

2.2 Monkey Tests and The Birthday Paradox 5

2.2.1 The Birthday Paradox Connection 6

2.2.2 Approximations vs Exact Distributions 6

2.3 OPSO, OQSO, and DNA Test Definitions 7

2.3.1 OPSO: Overlapping Pairs Sparse Occupancy 7

2.3.2 OQSO: Overlapping Quadruples Sparse Occupancy . 7

2.3.3 The DNA Test 8

2.4 Random Walks and Spectral Graph Theory 8

2.4.1 The Complete Directed Graph 8

2.4.2 Missing Edges as a Subgraph 9

2.4.3 The Transfer Matrix Method 9

2.5 Finite Automata and Formal Languages 9

2.5.1 Deterministic Finite Automata (DFA) 10

2.5.2 The Product Construction and Intersection 10

2.5.3 Matrix Representation of DFAs 11

2.6 Equivalence Relations 11
2.7 Combinatorial Preliminaries 12
2.7.1 Set Partitions and Bell Numbers 12
2.7.2 Restricted Growth Strings (RGS) 12
2.8 Linear Algebra and Matrix Exponentiation 13
2.8.1 The Cayley-Hamilton Theorem 13
2.8.2 Polynomial Modular Arithmetic Optimization 14

3 Related Work **15**
3.1 Automata- and Matrix-Based Analysis of OPSO 15
3.2 High Precision Numerical Analysis of Monkey Tests 16
3.3 Gen. Func. & Asymp. Analysis 18
3.4 Missing Words, Nullomers, and Applications 19
3.4.1 Nullomers in Genomics 19
3.4.2 Data Sparsity in Language Modeling 19
3.4.3 Missing Patterns in Cryptographic Test Suites 20

4 The Random Variable: Number of Missing Edges **21**

5 Automata Based Monitoring **23**

6 Counting Paths with Edge Omission **26**
6.1 Matrix Exponentiation 26
6.2 Cayley-Hamilton Based Computation 27
6.3 Complexity Analysis 28
6.3.1 Time Complexity of Matrix Exponentiation 28
6.3.2 Time Complexity of Cayley-Hamilton Method 28
6.3.3 Space Complexity 29

7 Equivalence Classes of Edge Tuples **30**

8 Representatives of Equivalence Classes **33**

9 Moments of the Omitted Edge Count **36**

10 Evaluation: The OPSO Case Study **38**

10.1 Numerical Results 39

10.1.1 Computational Performance 39

10.1.2 Accuracy and Moment Estimation 40

10.2 Comparison with Related Methods 41

11 Discussion **44**

11.1 Positioning Relative to Related Work 44

11.2 Interpretation of Results 45

11.3 Strengths of the Automata-Theoretic Approach 45

11.4 Scalability Limits and Bell Numbers 45

11.5 Statistical Meaning of Higher Moments 46

11.6 Future Directions and Applications 46

11.6.1 Extensions of the Framework 46

11.6.2 Other Possible Applications of the Framework 46

11.7 Conclusion 47

A Implementation Notes **49**

A.1 Software Structure and Implementation 49

A.2 Reproducibility 50

1 Introduction

In this work, we study the following problem. Given a random word of length $L + 1$ over an alphabet of size n, we seek to determine the expectation, variance, and higher moments of the number of *missing* 2-grams, that is, pairs of consecutive symbols that do not appear in the word. Equivalently, this can be viewed as counting the number of missing edges in a random walk of length L over the complete directed graph with vertex set $[n]$ and edge set $[n]^2$ (including self loops).

Rahmann and Rivals [8] developed a method for computing the expectation and variance of this random variable by partitioning edge configurations into equivalence classes based on graph isomorphism. For each class, they use generating functions and correlation matrices to count the number of valid paths. Their approach, however, does not extend directly to higher moments, as it would require a characterization and efficient enumeration of $m \times m$ matrices of correlation vectors; they note significant obstacles to extending to higher moments. They actually state this as an open problem.

While the problem of missing words generalizes the classical urn problem, the presence of overlaps between consecutive words introduces complex dependencies that make exact analysis notoriously difficult. Standard combinatorial approaches, such as inclusion-exclusion or generating functions, effectively handle the expectation and variance but struggle to extend beyond the first few moments. This is due to the combinatorial explosion of overlapping configurations, which requires characterizing an increasingly complex hierarchy of correlation matrices. Addressing this challenge requires a fundamental shift in perspective: from enumerating explicit correlation patterns to modeling the state-based process that generates them.

We follow a similar strategy of exploiting the inherent symmetries of the complete graph to partition configurations into equivalence classes. How-

ever, instead of using generating functions, we solve the core counting problem by constructing a deterministic finite automaton (DFA) for each unique class representative. By computing the powers of the automaton's transition matrix, we obtain the exact count of valid paths, which allows us to aggregate these counts, weighted by the size of their respective equivalence classes, to derive the exact moments of the missing edge distribution.

This thesis provides formal definitions, constructions, and proofs of correctness for every stage of this process. We also analyze the computational complexity of the overall approach and show that it can be implemented efficiently. Furthermore, we implement and evaluate a fully analytical generating function method for the first two moments, following and reproducing approaches from prior work, in particular that of Leopardi [6], in order to enable direct comparison. Finally, we demonstrate the practical applicability of the method by using it to compute, within seconds of CPU time, a previously unknown numerical quantity needed for the evaluation of random generators [7].

The remainder of the thesis is organized as follows. Chapter 2 introduces the necessary preliminaries and background material. Chapter 3 discusses related work and places our contribution in context. In Chapter 4, we formalize the problem and establish the theoretical foundation. In Chapter 5, we present the automaton construction used to monitor path violations, and Chapter 6 details the matrix-based techniques for efficient computation of path counts. To manage the complexity of the state space, Chapter 7 introduces an equivalence relation over edge sets that groups isomorphic configurations, while Chapter 8 provides an algorithm for generating canonical representatives. These components are integrated in Chapter 9 to compute the aggregated moment statistics, which are subsequently applied to the OPSO test in Chapter 10. Chapter 11 discusses the implications of our findings, outlines potential extensions and applications of the framework, and concludes the thesis.

2 Preliminaries and Background

This chapter establishes the mathematical definitions and notations that will be used throughout the thesis. We cover fundamental concepts in probability theory, formal languages, graph theory, and combinatorics. These tools provide the necessary framework for understanding the specialized automata construction and moment calculations presented in subsequent chapters.

2.1 Probability Background

We begin by recalling the basic definition of moments, which quantify the shape of a probability distribution. Let X be a discrete random variable taking values in a countable set $\Omega \subseteq \mathbb{R}$, with probability mass function $p(x) = \Pr(X = x)$. The *expectation* (or first moment) of X, denoted $\mathbb{E}[X]$, represents the central tendency of the distribution and is defined as:

$$\mathbb{E}[X] = \sum_{x \in \Omega} x \cdot p(x).$$

The m-th *raw moment* of X is defined as:

$$\mathbb{E}[X^m] = \sum_{x \in \Omega} x^m \cdot p(x).$$

2.1.1 Standardized Moments: Skewness and Kurtosis

Higher moments provide increasingly detailed information about the distribution. While expectation and variance are standard, the third and fourth moments are critical for understanding the "tails" of a distribution, which is essential for high-precision randomness testing.

- **Variance**: The second central moment, $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$, measures the dispersion or spread of the distribution around the mean.

- **Skewness**: The third standardized moment, $\gamma_1 = \mathbb{E}[(\frac{X-\mu}{\sigma})^3]$, measures the asymmetry of the distribution about its mean. A positive skew implies a long right tail. In the context of missing patterns, a positive skew would indicate that while the average number of missing patterns is low, there is a non-negligible probability of observing a very large number of missing patterns.

- **Kurtosis**: The fourth standardized moment, $\mathrm{Kurt}[X] = \mathbb{E}[(\frac{X-\mu}{\sigma})^4]$, measures the "tailedness" or propensity for outliers. High kurtosis implies a distribution with heavy tails (leptokurtic). This is particularly relevant for OPSO, as we are interested in rare events. A high kurtosis warns us that extreme deviations from the mean are more likely than a Normal approximation would suggest.

Real-World Analogy: Financial Markets. To intuit the importance of these moments, consider stock market returns. The *mean* tells you the average daily return (e.g., +0.05%), and the *variance* tells you the volatility (how much it fluctuates). However, assuming returns follow a Normal distribution (which has 0 skew and 3 kurtosis) can be disastrous. A negative *skewness* implies that while most days are slightly positive, "bad" days are much worse than "good" days. A high *kurtosis* ("fat tails") implies that extreme events, like the 1987 Black Monday crash, occur far more frequently than a Normal model predicts (e.g., once a decade vs. once in the age of the universe). In randomness testing, a PRNG plays the role of the "market": if the distribution of the test statistic exhibits high kurtosis, a rare but legitimate extreme deviation may lead to the false rejection of an otherwise good generator.

Why are expectations insufficient? Tests like OPSO rely on comparing the observed count to a theoretical distribution. Often, a Normal approximation is used ($Z = \frac{X-\mu}{\sigma}$). However, if the underlying distribution has significant skewness or kurtosis, the Normal approximation fails in the tails, leading to incorrect p-values and potential false positives (rejecting a good RNG) or false negatives (accepting a bad one). Exact computation of higher moments allows for more robust testing via Edgeworth expansions or direct Moment-Bound comparisons.

2.1.2 Linearity of Expectation and Indicator Variables

The driving engine of our initial derivations is the *Linearity of Expectation*. Consider a set of events $A_1, A_2, \ldots, A_N$. Let I_j be the indicator random variable for event A_j, such that $I_j = 1$ if A_j occurs and 0 otherwise. The total number of occurrences $X = \sum_{j=1}^{N} I_j$ has expectation:

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{j=1}^{N} I_j\right] = \sum_{j=1}^{N} \mathbb{E}[I_j] = \sum_{j=1}^{N} \Pr(A_j).$$

This holds *regardless* of the dependence structure between the events A_j. In the context of the OPSO test, let I_w be the indicator that the word w is missing from the sequence. The expected number of missing words is simply the sum of the probabilities that each individual word w is missing.

However, linearity does *not* simplify the calculation of higher moments.

$$\mathbb{E}[X^2] = \mathbb{E}\left[\left(\sum_i I_i\right)\left(\sum_j I_j\right)\right] = \sum_{i,j} \mathbb{E}[I_i I_j] = \sum_{i,j} \Pr(A_i \cap A_j).$$

Computing the second moment requires summing the probabilities of observing *pairs* of events simultaneously (correlations). Generalizing to the m-th moment requires summing probabilities of intersections of m events:

$$\mathbb{E}[X^m] = \sum_{w_1,\ldots,w_m} \Pr(w_1 \text{ is missing } \wedge \cdots \wedge w_m \text{ is missing}).$$

This combinatorial expansion is where the complexity lies. As m increases, the number of terms grows as N^m, and the dependency structure between overlapping words becomes intricate. Our thesis solves this by classifying these m-wise intersections using automata and symmetry groups.

2.2 Monkey Tests and The Birthday Paradox

The term "Monkey Tests" derives from the "Infinite Monkey Theorem", which posits that a monkey hitting keys randomly on a typewriter given infinite time will almost surely reproduce any given text, such as the Bible. In the context of pseudo-random number generator (PRNG) testing, this metaphor is inverted: we observe a *finite* sequence produced by a generator and ask whether the frequency of occurrence of different substrings patterns matches what we would expect from a purely random source (a "perfect monkey").

2.2.1 The Birthday Paradox Connection

The statistical power of tests like OPSO (Overlapping Pairs Sparse Occupancy) relies on the principles behind the *Birthday Paradox*. In the classic problem, we ask for the probability that two people in a room of k individuals share a birthday, assuming $n = 365$ days. The probability of *no* collision (all birthdays distinct) is:

$$P(\text{no match}) = \frac{n!}{n^k(n-k)!} \approx e^{-\frac{k^2}{2n}}.$$

When $k \approx \sqrt{n}$, the probability of a collision becomes significant ($\approx 50\%$). In our context, the "days" are the $M = 2^{20}$ possible word patterns, and the "people" are the $L = 2^{21}$ overlapping words in our sequence. Since $L \approx 2M$, we are well past the $\sqrt{M}$ threshold; we are in a regime where we expect many collisions (repeated words) and, conversely, many empty cells (missing words).

If $L \ll M \ln M$ (the Coupon Collector threshold), we expect many patterns to remain unobserved (missing). The OPSO test operates in the "sparse occupancy" regime where the number of samples L is of the same order of magnitude as the sample space M. In this regime, the number of missing words provides a sensitive statistic. If a generator is biased, it "prefers" certain values, causing more collisions than expected and leaving more cells empty. Thus, an abnormally high number of missing words is a hallmark of a defective generator.

2.2.2 Approximations vs Exact Distributions

Standard results in probability theory, such as those obtained via the Chen-Stein method, suggest that for large M and L, the number of missing words converges to a Poisson or Normal distribution. Specifically, if independence is assumed, the probability that a specific bin is empty is $q = (1 - 1/M)^L \approx e^{-L/M}$. The expected number of missing words is $\mu = Me^{-L/M}$. However, for overlapping words, the trials are *not* independent. The occurrence of 'AB' affects the probability of 'BC'. While Poisson approximations are often "good enough" for simple pass/fail certifications, they break down in the tails. For high-assurance cryptography or rigorous scientific simulation, relying on approximations introduces methodological error. Our work replaces these approximations with exact combinatorial counts, allowing for perfect calibration of the test statistic.

2.3 OPSO, OQSO, and DNA Test Definitions

The Marsaglia Diehard Battery [7] is a collection of statistical tests for measuring the quality of a random number generator. It includes several "Monkey Tests" that focus on the distribution of missing words. We focus on three specific tests that share the same underlying parameters but differ in alphabet size and word length.

2.3.1 OPSO: Overlapping Pairs Sparse Occupancy

The OPSO test views the random stream as a sequence of integer pairs.

Definition 2.1 (OPSO Parameters).

- **Alphabet**: $\Sigma = \{0, 1, \ldots, 1023\}$ (size $n = 2^{10} = 1024$).
- **Word Length**: 2-grams (pairs of integers).
- **State Space Size**: There are $n^2 = (2^{10})^2 = 2^{20} = 1,048,576$ possible pairs.
- **Sequence Length**: The test consumes $2^{21} + 1 = 2,097,153$ keystrokes (approximately 2 million).

The test considers all 2^{21} overlapping pairs in the sequence and counts how many of the 2^{20} possible pairs *do not* appear.

Note the ratio $L/n^2 = 2^{21}/2^{20} = 2$. On average, we expect each pair to appear about 2 times. However, due to randomness, many will typically appear 4, 5, or more times, while approximately 13.5% (e^{-2}) will not appear at all.

2.3.2 OQSO: Overlapping Quadruples Sparse Occupancy

The OQSO test is structurally identical to OPSO but operates on quadruples of bytes.

- **Alphabet**: $\Sigma = \{0, \ldots, 31\}$ (size $n = 32 = 2^5$).
- **Word Length**: 4-grams.
- **State Space**: $n^4 = (32)^4 = 2^{20}$.

Notice that the total number of patterns is again 2^{20}, matching OPSO. Consequently, the current framework can be extended to OQSO, with the primary structural change occurring in the construction of the underlying DFA.

2.3.3 The DNA Test

The DNA test mimics the analysis of genetic sequences.

- **Alphabet**: $\Sigma = \{C, G, A, T\}$ (size $n = 4 = 2^2$).
- **Word Length**: 10-grams (sequences of 10 bases).
- **State Space**: $n^{10} = 4^{10} = 2^{20}$.

Again, the state space size is preserved at 2^{20}. This consistency across tests is intentional: it standardizes the statistical power and expected number of collisions across different sparsity regimes. Because all three tests share the state space size $M = 2^{20}$ and sequence length $L = 2^{21}$, they are mathematically isomorphic from the perspective of our general framework (counting missing paths in a graph), differing only in the mapping from random bits to graph vertices.

Table 2.1: Parameters of standard Monkey Tests in the Diehard Battery.

Test	**Alph. Size (n)**	**Word Len. (m)**	**Pattern Space (M)**	**# Words (L)**
OPSO	1024 (2^{10})	2	2^{20}	2^{21}
OQSO	32 (2^5)	4	2^{20}	2^{21}
DNA	4 (2^2)	10	2^{20}	2^{21}

2.4 Random Walks and Spectral Graph Theory

Our problem can be rigorously framed in the language of Graph Theory. A sequence of random symbols $v_1, v_2, \ldots, v_{L+1}$ generated uniformly from $\Sigma = \{1, \ldots, n\}$ corresponds to a *Random Walk* on a graph.

2.4.1 The Complete Directed Graph

Let $G_n = (V, E)$ be the complete directed graph with self-loops on n vertices.

- **Vertices** $V = \Sigma$: Each vertex represents a symbol in the alphabet.
- **Edges** $E = V \times V$: There is a directed edge from every vertex u to every vertex v, including self-loops (u, u).

Traversing the edge (u, v) corresponds to the symbol pair '...u v ...' appearing in the sequence. A sequence of length $L + 1$ corresponds to a walk of L steps (edges).

2.4.2 Missing Edges as a Subgraph

The count of "missing 2-grams" is exactly the number of edges in G_n that were *never traversed* by the random walk. If we forbid a specific set of edges $S \subset E$, asking "what is the probability that no edge in S appears in the walk?" is equivalent to asking "what is the number of walks of length L in the subgraph $G' = (V, E \setminus S)$?".

2.4.3 The Transfer Matrix Method

The use of matrix powers to count paths in graphs is a standard technique in enumerative combinatorics, known as the *Transfer Matrix Method*. As detailed by Stanley [11], the number of walks of length L between vertices i and j in a graph with adjacency matrix A is given by the (i, j) entry of A^L. For the complete graph G_n, the adjacency matrix acts as the all-ones matrix J. For the subgraph G' avoiding edges S, the effective adjacency matrix T_S is $J - I_S$, where I_S is the incidence matrix of the forbidden set. Computing the eigenvalues and eigenvectors of this matrix T_S allows us to express the total count of valid paths as a sum of powers of eigenvalues:

$$N(L) = \sum_{k=1}^{n} c_k \lambda_k^L.$$

Our specific automaton construction essentially builds a highly efficient version of this matrix T_S, often compressing it significantly by exploiting the specific structure of S.

2.5 Finite Automata and Formal Languages

To formalize the constraint of "avoiding a set of edges," we turn to *Formal Language Theory*. The set of all valid sequences constitutes a *Regular Language*, which can be recognized by a Finite Automaton.

2.5.1 Deterministic Finite Automata (DFA)

A Deterministic Finite Automaton is a computational model consisting of states and transitions. Formally, it is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$:

- Q: A finite set of internal states.
- Σ: The input alphabet.
- $\delta : Q \times \Sigma \to Q$: The transition function, determining the next state based on the current state and input symbol.
- $q_0 \in Q$: The initial state.
- $F \subseteq Q$: The set of accepting states (sequences ending in these states are "valid").

In our specific application, the states of the DFA encode the "memory" of the walk. Since we are avoiding 2-grams (edges), the automaton only needs to remember the *most recent vertex visited*. Thus, the state space corresponds roughly to the set of vertices V, plus a special "sink" state $\perp$ representing a violation (traversing a forbidden edge).

2.5.2 The Product Construction and Intersection

While counting strings avoiding a single pattern is simple (KMP algorithm / Aho-Corasick), we need to avoid *multiple* patterns simultaneously when computing higher moments. In language theory, if L_1 and L_2 are regular languages accepted by M_1 and M_2, then their intersection $L_1 \cap L_2$ is also regular. It is accepted by the *Product Automaton* $M = M_1 \times M_2$, whose states are pairs (q_1, q_2). This is implicitly what happens when we define a set of missing edges $S = \{e_1, e_2, \ldots, e_m\}$. We are looking for the intersection of the languages $L_{\text{avoid } e_1} \cap \cdots \cap L_{\text{avoid } e_m}$. However, constructing the full product leads to an exponential blowup in the number of states. A key contribution of this thesis is constructing the intersection automaton *directly* and efficiently, exploiting the fact that our patterns are simple length-2 edges, which prevents the state explosion typical of general regular expressions.

2.5.3 Matrix Representation of DFAs

Any DFA can be represented by its *Transition Matrix* T, an adjacency matrix of the state graph where T_{ij} counts the number of symbols causing a transition from q_i to q_j:

$$T_{ij} = |\{\sigma \in \Sigma \mid \delta(q_i, \sigma) = q_j\}|.$$

Crucially, the number of accepted strings of length L is given by:

$$N(L) = \mathbf{u}^\top T^L \mathbf{v},$$

where $\mathbf{u}$ is the initial state vector (one-hot encoded) and $\mathbf{v}$ is the indicator vector for the accepting states F. This identity allows us to perform all counts using matrix arithmetic rather than naive string enumeration.

2.6 Equivalence Relations

One of the main tools we use to manage combinatorial complexity is the concept of *equivalence relations*. A binary relation $\sim$ on a set X is an equivalence relation if it satisfies three properties for all $a, b, c \in X$:

- **Reflexivity**: $a \sim a$.
- **Symmetry**: If $a \sim b$, then $b \sim a$.
- **Transitivity**: If $a \sim b$ and $b \sim c$, then $a \sim c$.

An equivalence relation partitions the set X into disjoint subsets called *equivalence classes*. The equivalence class of an element x, denoted $[x]$, consists of all elements related to x:

$$[x] = \{y \in X \mid x \sim y\}.$$

The set of all equivalence classes is denoted $X/\sim$. A fundamental property of this partition is that any value computed over the entire set X can be computed by summing over the classes instead. Specifically, if a function $f : X \to \mathbb{R}$ is *class invariant* (i.e., $x \sim y \implies f(x) = f(y)$), then the total sum can be grouped as:

$$\sum_{x \in X} f(x) = \sum_{C \in X/\sim} |C| \cdot f(x_C),$$

where x_C is any representative element of the class C. This identity is crucial when $|X|$ is too large to iterate over, but the number of classes $|X/\sim|$ is manageable. In such cases, we solve the problem by iterating over canonical representatives x_C, computing the class size $|C|$, and evaluating $f(x_C)$.

2.7 Combinatorial Preliminaries

Our method relies heavily on structuring the state space of edge tuples. We employ standard combinatorial structures from the theory of partitions.

2.7.1 Set Partitions and Bell Numbers

A *partition* of a set S is a collection of disjoint non-empty subsets (blocks) whose union is S. For studying the structure of edge tuples, we essentially partition the $2m$ vertices involved in the m edges into groups that "must be equal".

- **Stirling Numbers**: The number of ways to partition a set of n elements into k blocks is the *Stirling number of the second kind*, $\left\{ {n \atop k} \right\}$.
- **Bell Numbers**: The total number of partitions of a set of size n is the *Bell number*, $B_n = \sum_{k=0}^{n} \left\{ {n \atop k} \right\}$.

For m forbidden edges, we involve $2m$ endpoints. The number of ways to equate these endpoints (defining the graph topology) is related to B_{2m}. The Bell numbers satisfy Dobiński's formula (connecting to moments of a Poisson distribution):

$$B_n = \frac{1}{e} \sum_{k=0}^{\infty} \frac{k^n}{k!}.$$

Asymptotically, $B_n \sim \frac{1}{\sqrt{n}} \left(\frac{n}{W(n)} \right)^{n+1/2} e^{\frac{n}{W(n)} - n - 1}$, where W is the Lambert W function. This superexponential growth ($B_{16} \approx 10^{10}$) represents the theoretical ceiling of our approach for very high moments.

2.7.2 Restricted Growth Strings (RGS)

To computationally enumerate partitions, we need a distinct encoding. We use *Restricted Growth Strings* (RGS). A sequence $w = w_1 w_2 \ldots w_n$ is an RGS

if $w_1 = 1$ and $w_i \leq \max(w_1 \dots w_{i-1}) + 1$. There is a direct bijection between Set Partitions and RGSs:

- The value w_i is the index of the block containing the i-th element.
- The "restricted growth" property ensures a canonical numbering of blocks (block 1 appears first, then block 2, etc.), preventing duplicates due to block relabeling.

For example, the partition $\{\{1,3\},\{2\}\}$ of $\{1,2,3\}$ maps to RGS 121 ($1 \in B_1, 2 \in B_2, 3 \in B_1$). The string 123 maps to $\{\{1\},\{2\},\{3\}\}$. In Chapter 8, we adapt an algorithm by Ruskey to generate all RGSs of length $2m$ efficiently. This serves as the "driver" loop for our entire computation, iterating through all distinct graph topologies.

2.8 Linear Algebra and Matrix Exponentiation

The computational core of our work is evaluating $N = \mathbf{u}^\top M^L \mathbf{v}$ for a large sparse matrix M and huge L (2^{21}). Naive matrix multiplication $M \times M$ takes $O(k^3)$ time for a $k \times k$ matrix. Computing M^L via binary exponentiation (Square-and-Multiply) takes $O(k^3 \log L)$. While $\log L = 21$ is small, the matrix dimension k grows with the number of forbidden edges. Moreover, the entries of M^L grow exponentially, leading to expensive multi-precision arithmetic.

2.8.1 The Cayley-Hamilton Theorem

We can significantly optimize this using the *Cayley-Hamilton Theorem*, which states that every square matrix satisfies its own characteristic equation.

Theorem 2.1 (Cayley-Hamilton). *Let A be an $k \times k$ matrix and let $p(\lambda) = \det(\lambda I - A) = \lambda^k + c_{k-1}\lambda^{k-1} + \cdots + c_0$ be its characteristic polynomial. Then:*

$$p(A) = A^k + c_{k-1}A^{k-1} + \cdots + c_0 I = 0.$$

This implies that A^k (and any higher power A^t for $t \geq k$) can be expressed as a linear combination of the lower powers $\{I, A, \dots, A^{k-1}\}$. Specifically, the powers of A span a *Krylov Subspace* of dimension at most k.

2.8.2 Polynomial Modular Arithmetic Optimization

Instead of computing the matrix power M^L directly, we operate in the polynomial ring $\mathbb{Q}[x]/\langle p(x)\rangle$. The algorithm proceeds as follows:

1. **Characteristic Polynomial**: Compute $p(x) = \det(xI - M)$. Since M is small and structured, this is fast using symbolic algorithms (e.g., Berkowitz algorithm).

2. **Scalar Exponentiation**: Compute the scalar polynomial remainder $r(x) = x^L \pmod{p(x)}$.

$$r(x) = r_{k-1}x^{k-1} + \cdots + r_1 x + r_0.$$

   This step effectively "fast-forwards" the recurrence relation defined by the matrix. It is performed using *scalar* arithmetic, which is practically instantaneous compared to matrix arithmetic.

3. **Matrix Evaluation**: Substitute M back into the remainder polynomial:

$$M^L = r(M) = \sum_{i=0}^{k-1} r_i M^i.$$

Complexity Gain: The prohibitive $O(k^3 \log L)$ matrix multiplications are replaced by $O(k^2 \log L)$ scalar polynomial operations (or $O(k \log k \log L)$ with FFT) and a final $O(k^4)$ evaluation. Since L is massive (10^6) and k is relatively small (< 100 for relevant automata), this trade-off yields a dramatic speedup, as shown in our results chapter. Furthermore, it keeps the numbers small (coefficients of $p(x)$) until the final recombination step, managing the bit-complexity explosion.

3 Related Work

The study of missing patterns in random sequences lies at the intersection of combinatorics, probability theory, and empirical testing of pseudorandom number generators (PRNGs). The phenomenon of missing short patterns arises in a variety of applied domains, including genomics, language modeling, and statistical validation of random and pseudorandom sources. In all these cases, the central question is whether a missing pattern reflects a genuine structural bias in the source, or whether it can be explained by chance alone, given that the sequence has finite length and consists of independent, uniformly distributed symbols.

The OPSO, OQSO, and related "Monkey" tests are central to practical randomness testing, yet their theoretical analysis has arguably lagged behind their widespread adoption. This chapter surveys the main lines of prior research relevant to our work, focusing on exact automata and matrix-based analysis, high-precision numerical evaluation of Monkey tests, asymptotic theory of missing words using generating functions, and applications in genomics and language modeling.

3.1 Automata- and Matrix-Based Analysis of OPSO

A major step toward exact, non-asymptotic analysis of the OPSO test was taken by Azran [2]. His MSc thesis, conducted under the supervision of Prof. Gera Weiss and Prof. Oded Margalit, constitutes the closest direct predecessor to the present work and established an automata- and matrix-based formulation for analyzing missing pattern statistics exactly.

The central contribution of this line of work is the observation that the absence of forbidden patterns can be modeled as a path counting problem on a finite automaton. By representing the automaton transition structure as a matrix, exact counts of valid sequences can be obtained via matrix exponentiation. This framework enables exact, non-asymptotic computation

of the expectation and variance of the OPSO statistic, replacing classical approximations with precise combinatorial counts.

The same formulation also shows, at a formal level, that higher moments can be expressed as sums of joint absence probabilities over multiple forbidden patterns. However, while this establishes theoretical computability, the thesis does not provide a complete constructive algorithm for evaluating higher order moments in practice. In particular, it does not address the systematic enumeration of combinatorial equivalence classes induced by alphabet relabelings, nor does it analyze the resulting complexity growth as the moment order increases. As a consequence, the practical scope of the work is limited to the first two moments.

This thesis adopts the same automata- and matrix-based viewpoint as its starting point but extends it in several essential directions. First, we provide a canonical and exhaustive enumeration of equivalence classes of forbidden pattern tuples using restricted growth strings, enabling higher moments to be computed systematically. Second, we give an explicit complexity analysis showing that the number of such classes grows according to the Bell numbers, thereby clarifying the true computational bottlenecks of the problem.

We found that transition matrices are not guaranteed to be diagonalizable (see Remark 6.1). We therefore avoid spectral methods in favor of a Cayley-Hamilton reduction, which applies effectively to any square matrix and guarantees exact results for all moment orders.

In this sense, the present work can be viewed as a strict generalization and algorithmic completion of the earlier automata-based framework. It retains the core modeling insight while extending it to a fully operational pipeline for computing arbitrary higher order moments, with well defined complexity guarantees and exact integer arithmetic throughout.

3.2 High Precision Numerical Analysis of Monkey Tests

A complementary line of work is due to Leopardi [6], whose focus is on the reliability and correctness of empirical randomness tests as implemented in practice, particularly within the TestU01 framework. Rather than proposing new tests, Leopardi examines the statistical models underlying existing tests and investigates the consequences of inaccuracies in their theoretical parameters.

Leopardi's work is motivated by the observation that many empirical tests

rely on approximate or asymptotic distributions of their test statistics. In the context of two-level testing, such approximations can lead to systematic false rejections of otherwise high quality PRNGs. This issue is particularly acute for the overlapping serial ("Monkey") tests, including OPSO, OQSO, and DNA, which are based on the number of missing words in a random string.

To address this, Leopardi revisits the theoretical computation of the expectation and variance of the Monkey test statistics. Building on generating function techniques due to Guibas and Odlyzko and on correlation polynomial methods developed by Rahmann and Rivals, Leopardi computes the exact expectation and variance for the OPSO, OQSO, and DNA tests to very high numerical precision, reporting results to approximately twenty decimal digits. These values are used to correct inaccuracies in existing TestU01 implementations and to demonstrate that previously observed test failures were often artifacts of incorrect theoretical parameters rather than deficiencies of the PRNGs under test.

A key technical contribution of Leopardi's work is the explicit enumeration of correlation classes of word pairs. Instead of summing over all pairs of words, which is computationally infeasible for large alphabets, words are grouped into equivalence classes determined by their overlap structure. For each class, the corresponding generating functions are evaluated, and the total contribution is obtained by multiplying by the class multiplicity. This approach enables exact computation of low order moments despite the large combinatorial space.

From the perspective of this work, Leopardi's work plays two important roles. First, it provides high precision numerical benchmarks for the expectation and variance of OPSO and related tests, against which exact symbolic or integer computations can be validated. Second, it highlights both the power and the limitations of generating function based approaches. While these methods provide fast and accurate results for the expectation and variance of missing word statistics, extending them beyond low order moments leads to substantial combinatorial difficulties, a limitation that is analyzed in detail in the work of Rahmann and Rivals [8].

The present work reproduces the first two moments reported by Leopardi exactly and goes beyond them by adopting a different representation of dependencies. Instead of explicitly characterizing correlation matrices between words, the proposed automata-based framework encodes pattern interactions implicitly through symmetry classes of forbidden transitions, enabling the exact computation of higher order moments in a systematic

and principled manner.

3.3 Generating Function and Asymptotic Analysis of Missing Words

The theoretical foundations of missing word statistics were laid by Rahmann and Rivals [8], who studied the distribution of the number of missing q-grams in random texts. Their work generalizes the classical occupancy problem of empty urns to the setting of overlapping words in random strings.

Rahmann and Rivals consider a random text of length $n + q - 1$ over an alphabet of size σ and define $X(n, \sigma, q)$ as the number of q-grams that do not occur in the text. For $q = 1$, this reduces to the well understood problem of empty urns. For $q \geq 2$, however, dependencies arise due to overlaps between successive words, making the analysis substantially more complex.

Using generating functions and correlation polynomials, Rahmann and Rivals derive exact expressions for the absence probability of a given word and asymptotic expressions for the expectation of $X(n, \sigma, q)$ in the regime where n and σ tend to infinity with fixed ratio $\lambda = n/\sigma^q$. A notable outcome of their analysis is that, asymptotically, the expected number of missing q-grams closely matches that of the independent urn model, despite the strong dependencies induced by overlaps.

The variance, however, is shown to be considerably more subtle. Rahmann and Rivals derive an explicit asymptotic formula for the variance in the case $q = 2$ and conjecture analogous forms for larger q. They emphasize that extending these results to higher moments requires one to characterize and enumerate correlation matrices between multiple words, and to count how many k-tuples of words with $k > 2$ induce each such matrix. This structure is not yet understood, in contrast to single word autocorrelations, and is identified as the central open problem preventing the computation of higher moments within the generating function framework.

Existing approaches, originating with Guibas and Odlyzko [4], attempt to characterize dependencies explicitly, either via correlation polynomials or correlation matrices between words. The central difficulty is that these objects grow combinatorially with the moment order and lack a known classification. In contrast, the automata-theoretic approach adopted here does not represent correlations explicitly. Instead, dependencies are en-

coded implicitly in the transition structure of a finite automaton, allowing higher-order interactions to be handled uniformly via path counting.

3.4 Missing Words, Nullomers, and Applications

3.4.1 Nullomers in Genomics

Acquisti et al. [1] study *nullomers*, defined as short DNA sequences that are completely absent from the human genome. The main result of their work is that the absence of many such short sequences does not necessarily imply biological lethality or negative selection. Instead, the authors show that mutational biases, in particular the high mutation rate of CpG dinucleotides, are sufficient to explain the observed pattern of missing words.

By analyzing the distribution of near matches to nullomers and comparing across multiple genomes, Acquisti et al. demonstrate that nullomers are often separated by only a small number of point mutations from highly frequent sequences. This provides strong evidence that their absence is compatible with neutral evolutionary processes rather than selective pressure.

The relevance of this prior research to our study lies in its reliance on a statistical baseline for pattern absence. Any claim that a word is missing due to selection requires an understanding of how likely such an absence is under a random model with comparable length and composition. The present work contributes to this foundational question by providing exact, non-asymptotic distributions of missing pattern statistics, which can serve as a rigorous reference point for assessing statistical significance in genomic applications.

3.4.2 Data Sparsity in Language Modeling

Chen and Goodman [3] provide an empirical study of smoothing techniques for n-gram language models, addressing the fundamental problem of data sparsity. The main result of their work is that effective language models must explicitly account for the large number of possible n-grams that are never observed, even in very large corpora, and that smoothing methods differ substantially in how they redistribute probability mass to these unseen events.

Their study shows that techniques such as Good Turing estimation and

interpolated models achieve superior performance by leveraging information about the expected number of unseen n-grams. Although the focus is on empirical language modeling, the underlying statistical question concerns the distribution of missing patterns as a function of sample size.

The connection to this work is conceptual rather than applied. While natural language is far from a uniform random source, the mathematical structure of omission probabilities under a random model provides a baseline against which deviations due to structure can be measured. Exact knowledge of higher order moments of the number of missing patterns clarifies the variability and tail behavior of omission counts, which is directly relevant to inference methods that rely on unseen events.

3.4.3 Missing Patterns in Cryptographic Test Suites

Rukhin et al. [9] describe the NIST Statistical Test Suite (SP 800–22), which provides a standardized collection of tests for assessing the quality of random and pseudorandom number generators used in cryptographic applications. A central result of this work is the formalization and widespread adoption of tests based on overlapping pattern frequencies, including OPSO, OQSO, and related tests.

The report emphasizes that statistical testing depends critically on the accuracy of the assumed null distributions of test statistics. In practice, however, many tests rely on asymptotic approximations or empirically fitted parameters. Subsequent analyses have shown that inaccuracies in these reference distributions can lead to systematic false rejections or incorrect confidence estimates.

This thesis directly addresses this issue by providing exact, non-asymptotic distributions for missing pattern statistics under idealized random models. By extending existing analyses beyond expectation and variance to arbitrary higher order moments, the work strengthens the theoretical foundation of these standardized tests and clarifies the reliability of their tail behavior.

4 The Random Variable: Number of Missing Edges

We analyze the statistical properties of the number of absent edges in paths traversing a fully connected directed graph. Consider a complete directed graph with self loops, denoted as $([n], [n]^2)$, where n is a large integer and $[n] = \{1, \ldots, n\}$. A random path of length L (sequence of $L + 1$ vertices) is a sequence $v_1, \ldots, v_{L+1}$, chosen uniformly at random from $[n]^{L+1}$.

We define the random variable X to be the number of edges in $[n]^2$ that do not appear as a consecutive pair (v_i, v_{i+1}) in the path. Using indicator variables X_e for each edge $e \in [n]^2$, where $X_e = 1$ if edge e is omitted and 0 otherwise, we have:

$$X = \sum_{e \in [n]^2} X_e.$$

Our objective is to determine not only the expectation of X, but also its higher moments. The expected number of omitted edges is:

$$\mathbb{E}[X] = \sum_{e \in [n]^2} \mathbb{E}[X_e] = \sum_{e \in [n]^2} \Pr[\text{edge } e \text{ is omitted}].$$

More generally, for any integer m, the m-th moment of X is given by:

$$
\begin{aligned}
\mathbb{E}[X^m] &= \mathbb{E}\left[\left(\sum_{e\in[n]^2} X_e\right)^m\right] \\
&= \sum_{\mathbf{e}\in([n]^2)^m} \mathbb{E}\left[\prod_{i=1}^{m} X_{e_i}\right] \\
&= \sum_{\mathbf{e}\in([n]^2)^m} \Pr[\text{all edges in } \mathbf{e} \text{ are omitted}] \\
&= \frac{1}{n^{L+1}} \sum_{\mathbf{e}\in([n]^2)^m} \#\{\text{paths of length } L \text{ omitting all edges in } \mathbf{e}\}
\end{aligned}
$$

This formulation sums over all ordered m-tuples of edges $\mathbf{e} = (e_1, \ldots, e_m)$. For any such tuple, the condition "all edges in $\mathbf{e}$ are omitted" depends only on the set of edges $Set(\mathbf{e}) = \{e_1, \ldots, e_m\}$ contained in the tuple. Thus, the contribution of a tuple $\mathbf{e}$ can be computed by evaluating $\text{PathCount}(n, L, Set(\mathbf{e}))$, as defined below:

Definition 4.1. Let $\text{PathCount}(n, L, S)$ denote the total number of paths of length L (sequences of $L+1$ vertices) in the complete directed graph $([n], [n]^2)$ whose edge sequence does not contain any edge from the set $S \subseteq [n]^2$.

This quantity is the building block for $\text{OmissionMoment}(n, L, m)$, which we use to denote the calculation of the aggregated moment. While brute force counting is viable for small n and L, we target efficient algorithms for large parameters, scaling with the moment order m.

5 Automata Based Monitoring

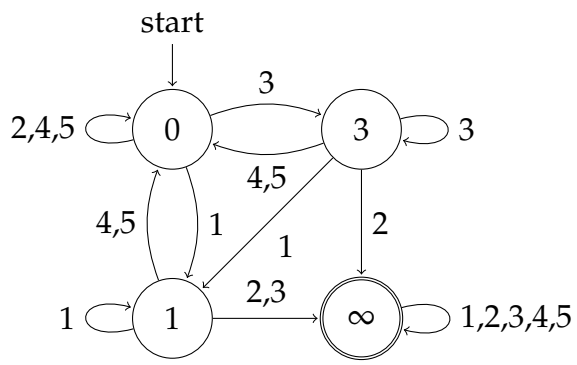
To compute $\mathrm{PathCount}(n, L, S)$ efficiently, we design an automaton monitoring path traversal. This automaton, $\mathrm{Aut}(n, S)$, employs a state space comprising a neutral state 0 for paths not immediately threatening to hit S, states labeled by source vertices of edges in S, and an absorbing state ∞ indicating set intersection.

Definition 5.1. For a set of edges $S \subseteq [n]^2$, let $U = \{u \colon (u, v) \in S\}$ be the set of source vertices. We define the deterministic finite automaton (DFA) $\mathrm{Aut}(n, S) = (Q, \Sigma, \delta, q_0, F)$ as follows:

- $Q = U \cup \{0, \infty\}$
- $\Sigma = [n]$
- $\delta(u, v) = \begin{cases} \infty & \text{if } u = \infty \text{ or } (u, v) \in S \\ v & \text{if } u \neq \infty, (u, v) \notin S \text{ and } v \in U \\ 0 & \text{otherwise} \end{cases}$
- $q_0 = 0$
- $F = \{\infty\}$.

To illustrate this construction, we display the automaton that monitors paths on alphabet $[5]$ that might traverse any of the edges $(1, 2)$, $(1, 3)$, or $(3, 2)$:

Example 5.1. The automaton $\mathrm{Aut}(5, \{(1, 2), (1, 3), (3, 2)\})$ is



Edge labels such as $\{2, 4, 5\}$ indicate transitions on any of those symbols. The transition matrix of this automaton (for any n, not only 5) is given by the following matrix, where the rows and columns are ordered as $0, 1, 3, \infty$:

$$\begin{pmatrix} n-2 & 1 & 1 & 0 \\ n-3 & 1 & 0 & 2 \\ n-3 & 1 & 1 & 1 \\ 0 & 0 & 0 & n \end{pmatrix}$$

The automaton demonstrates our path monitoring strategy: starting at state 0, it transitions to state 1 or 3 when these vertices are encountered, as they are sources of edges in S. From these states, if the next vertex completes a forbidden edge (for instance, reaching vertex 2 from state 1), the automaton moves to the absorbing state ∞. Otherwise, it either remains in these states (if one of these vertices is revisited) or returns to state 0 (if no potential threat exists).

The next proposition states the automaton's key property that enables efficient computation of path probabilities:

Proposition 5.1. *The automaton* $\mathrm{Aut}(n, S)$ *accepts a word* $v_1 \cdots v_{L+1} \in [n]^*$ *if and only if there exists* $1 \leq i \leq L$ *such that* $(v_i, v_{i+1}) \in S$.

Proof. We prove both directions of the equivalence.

First, assume $\mathrm{Aut}(n, S)$ accepts $w = v_1 \cdots v_{L+1}$. By definition of acceptance, there exists a sequence of transitions $q_0 \xrightarrow{v_1} q_1 \xrightarrow{v_2} \dots \xrightarrow{v_{L+1}} q_{L+1}$ where $q_{L+1} = \infty$. Since transitions to ∞ are permanent, $q_{L+1} = \infty$ implies that a forbidden transition occurred at some step $i \leq L$. The automaton first reaches state ∞ only when processing some vertex v_{i+1} from a state q_i where $(q_i, v_{i+1}) \in S$. Therefore, the path must traverse at least one edge from S, meaning $\{(v_1, v_2), (v_2, v_3), \dots, (v_L, v_{L+1})\} \cap S \neq \emptyset$.

Conversely, suppose $w = v_1 \cdots v_L$ contains an edge from S. Then there exists an index i where $(v_i, v_{i+1}) \in S$. After processing prefix $v_1 \cdots v_i$, the automaton must be either in state ∞ (ensuring acceptance) or in a state where reading v_{i+1} forces transition to ∞ by construction of δ. Thus, the automaton accepts w. $\square$

6 Counting Paths with Edge Omission

Having established the structure and behavior of the automaton, we now present two efficient algorithms for computing the number of valid paths. Specifically, given a natural number n, a length L, and a subset $S \subseteq [n]^2$, our goal is to compute the number of paths of length L (vertex sequence of length $L+1$) in the complete directed graph $([n], [n]^2)$ that avoids all edges in S.

This is accomplished by constructing the transition matrix

$$M = \text{TransitionMatrix}(\text{Aut}(n, S)),$$

which encodes the state transitions of the automaton $\text{Aut}(n, S)$ that accepts exactly those paths that avoid S. The computation then reduces to analyzing powers of M to evaluate the number of accepted paths of length L.

Remark 6.1. While the transition matrix M is not guaranteed to be diagonalizable in general (due to potential Jordan blocks associated with the zero eigenvalue), our methods do not rely on diagonalizability. The matrix exponentiation and Cayley-Hamilton approaches work for any square matrix.

6.1 Matrix Exponentiation

The first algorithm, shown in Algorithm 1, computes $\text{PathCount}(n, L, S)$ via direct matrix exponentiation. It raises the transition matrix M to the power $L+1$ and extracts the entry corresponding to the number of accepted paths. This returns the exact integer count of valid paths.

Algorithm 1 PathCount(n, L, S) using matrix exponentiation

Require: Alphabet size n, Path length L, Edge set S
Ensure: The number of paths of length L avoiding S
1: $M \leftarrow \text{TransitionMatrix}(\text{Aut}(n, S))$
2: **return** $n^{L+1} - M^{L+1}[1, -1]$ {Number of accepted paths}

We index the transition matrix M so that index 1 corresponds to the initial state 0, and the final index corresponds to the absorbing state ∞. Recall that for the transition matrix of a deterministic finite automaton, the entry $M^k[i, j]$ equals the number of words of length k that take the automaton from state i to state j. Since the automaton transitions from the initial state 0 to the absorbing state ∞ upon encountering a forbidden edge, and ∞ is absorbing, the entry $M^{L+1}[1, -1]$ counts exactly the number of paths of length L (sequences of $L + 1$ vertices) that contain at least one forbidden edge. The exponent is $L + 1$ because the automaton performs one transition for each vertex in the sequence: the first transition from the start state 0 consumes the first vertex v_1, and each subsequent transition consumes v_{i+1} based on the current state. Subtracting this quantity from the total number of paths n^{L+1} yields the number of valid paths that avoid all edges in S.

6.2 Cayley-Hamilton Based Computation

The second algorithm, shown in Algorithm 2, leverages the *Cayley-Hamilton theorem*, which states that every square matrix satisfies its own characteristic polynomial. This theorem allows us to express any high power M^k as a linear combination of the lower powers $I, M, \ldots, M^{m-1}$, where m is the size of the matrix. As a result, instead of computing large matrix powers explicitly, we can reduce the problem to *polynomial arithmetic modulo the characteristic polynomial*, substantially reducing the computational burden for large L.

Algorithm 2 begins by constructing the transition matrix M of the automaton that avoids transitions in S. It then computes the characteristic polynomial p_{char} of M, and uses repeated squaring and multiplications with polynomial modular reduction to efficiently compute a matrix polynomial p such that $p(M) \equiv M^{L+1} \mod p_{\text{char}}$. The final count is extracted from the appropriate matrix entry. Note that the bit-length of the polynomial coefficients grows linearly with L.

Algorithm 2 PathCount(n, L, S) using Cayley-Hamilton's theorem

Require: Alphabet size n, Path length L, Edge set S
Ensure: The number of paths of length L avoiding S

```
M ← TransitionMatrix(Aut(n, S))
p_char ← CharPoly(M)                    {Characteristic polynomial of M}
p ← 1;   q ← x                          {Initialize result p = 1, base q = x}
k ← L + 1
while k > 0 do
    if k is odd then
        p ← (p · q) mod p_char
    end if
    q ← (q · q) mod p_char
    k ← ⌊k/2⌋
end while
return n^(L+1) − p(M)[1, −1]            {Number of accepted paths}
```

6.3 Complexity Analysis

6.3.1 Time Complexity of Matrix Exponentiation

The *matrix exponentiation method* relies on computing M^{L+1} via binary exponentiation (square and multiply). This approach requires $O(\log L)$ matrix multiplications. Since each matrix multiplication involves $O(m^3)$ scalar multiplications, and the number of bits to keep the numbers grows to $O(L \log n)$, the total complexity is dominated by the cost of the final multiplications involving the largest numbers. Let $\mu(b)$ denote the bit-complexity of multiplying two b-bit integers (see Harvey and van der Hoeven [5]). The total time complexity is:

$$\boxed{O(m^3 \cdot \mu(L \log n))}$$

Note that while there are $O(\log L)$ steps, the bit-lengths in earlier steps are smaller (forming a geometric series), so the cost is dominated by the final step.

6.3.2 Time Complexity of Cayley-Hamilton Method

The *Cayley-Hamilton based method* avoids computing high powers of the matrix directly. Instead, it works with polynomials of degree m. First,

computing the characteristic polynomial p_{char} takes $O(m^4)$ time. Then, calculating $x^{L+1} \bmod p_{\text{char}}$ requires $O(\log L)$ polynomial squarings. Each squaring of a degree m polynomial takes $O(m \log m)$ scalar operations (using FFT) or $O(m^2)$ classically. Finally, evaluating the resulting polynomial $p(M)$ takes $O(m^4)$. The bit-complexity of the coefficients in the polynomial modular exponentiation grows much slower than the matrix entries in the direct method. The dominant factor for large L is the scalar polynomial arithmetic:

$$\boxed{O(m^2 \log L \cdot \mu(\text{poly coeff bits}) + m^4)}$$

Crucially, the dependence on L is only logarithmic in the number of arithmetic operations, making this method far superior for very large L when m is small.

6.3.3 Space Complexity

Both algorithms must store matrices of size $m \times m$, requiring at least $O(m^2)$ space. The matrix exponentiation method reuses matrices during repeated squaring and thus requires $O(m^2)$ space in practice. The Cayley-Hamilton based method, however, needs to maintain a linear combination of up to m powers of M when evaluating a degree-m polynomial. This requires storing up to m such matrices, leading to: $O(m^3)$ space.

For specific implementation details, the software architecture, and reproducibility guides, we refer the reader to Appendix A.

7 Equivalence Classes of Edge Tuples

Our next step is to compute the expected probability over all possible tuples of edges $\mathbf{e}$ with size m (some edges may be repeated). We denote this expected probability as $\text{OmissionMoment}(n, L, m)$.

Directly applying the method from the previous section to all possible tuples of edges is computationally infeasible due to the vast number of combinations. However, the structure of the automaton $\text{Aut}(n, Set(\mathbf{e}))$ depends only on the isomorphism class of the induced edge set $Set(\mathbf{e})$, not on the specific vertex labels. By grouping essentially identical forbidden tuples, we can drastically reduce the number of required computations.

While the number of possible tuples of edges is large, we only need to consider the different graphs that can be formed by the edges in $Set(\mathbf{e})$. This is because the automaton's behavior is determined by the structure of the graph rather than the specific edges themselves. Therefore, we can group tuples of edges that lead to the same automaton behavior and compute the expected probability for each unique graph structure.

To this end, we propose a classification of edge tuples based on their structural similarities. This classification is crucial for efficient computation, as it allows us to analyze multiple edge tuples simultaneously when they share equivalent behavior under our automaton construction:

Definition 7.1. Let $\mathbf{e} = (e_1, \ldots, e_m)$ and $\mathbf{f} = (f_1, \ldots, f_m)$ be two ordered m-tuples of edges in $([n]^2)^m$. We say that $\mathbf{e} \sim \mathbf{f}$ if there exists a vertex permutation $\pi \in [n]!$ such that applying π component-wise to $\mathbf{e}$ yields $\mathbf{f}$. That is, for all $k \in \{1, \ldots, m\}$, if $e_k = (u, v)$, then $f_k = (\pi(u), \pi(v))$. Let $Set(\mathbf{e}) = \{e_1, \ldots, e_m\} \subseteq [n]^2$ denote the set of edges induced by the tuple $\mathbf{e}$. Note that while $\mathbf{e}$ has length m, the size of $Set(\mathbf{e})$ may be smaller if $\mathbf{e}$ contains duplicates.

We prove that this relation is an equivalence relation, which allows us to partition the set of all possible edge tuples into equivalence classes:

Proposition 7.1. *The relation $\sim$ is an equivalence relation.*

Proof. The relation $\sim$ is reflexive because a tuple is similar to itself by the identity permutation. It is symmetric because if $\mathbf{e}$ is similar to $\mathbf{f}$ by a permutation π, then $\mathbf{f}$ is similar to $\mathbf{e}$ by the inverse permutation π^{-1}. It is transitive because if $\mathbf{e}_1$ is similar to $\mathbf{e}_2$ by a permutation π_1 and $\mathbf{e}_2$ is similar to $\mathbf{e}_3$ by a permutation π_2, then $\mathbf{e}_1$ is similar to $\mathbf{e}_3$ by the composition of permutations $\pi_2 \circ \pi_1$. □

This equivalence relation partitions edge tuples into classes that behave identically under vertex relabeling:

Proposition 7.2. *If $\mathbf{e} \sim \mathbf{f}$, then*

$$\text{PathCount}(n, L, Set(\mathbf{e})) = \text{PathCount}(n, L, Set(\mathbf{f}))$$

for every n and L.

Proof. The induced edge sets $Set(\mathbf{e})$ and $Set(\mathbf{f})$ create isomorphic graphs under the permutation π that witnesses $\mathbf{e} \sim \mathbf{f}$. Therefore, the automata $\text{Aut}(n, Set(\mathbf{e}))$ and $\text{Aut}(n, Set(\mathbf{f}))$ are isomorphic (modulo state relabeling). This preserves the structure of the transition matrix and the number of accepted paths, ensuring the omission probabilities are identical. □

While different equivalence classes may yield isomorphic automata, this particular decomposition offers two key advantages: it admits an efficient polynomial time indexing scheme, and it allows precise computation of equivalence class sizes. These properties are crucial for our subsequent probability calculations.

The size of each equivalence class is determined by counting how many distinct ways the vertices in an edge tuple $\mathbf{e}$ can be relabeled by permutations of $[n]$. Since only the labels of the vertices that actually appear in the induced set $Set(\mathbf{e})$ affect the structure of the edge set, the size of the equivalence class depends on the number of distinct vertices present in $Set(\mathbf{e})$. The count goes by observing that, for each such tuple, we can assign any subset of $|U|$ distinct labels from $[n]$ to the vertices in U, and the remaining $n - |U|$ vertices can be labeled arbitrarily. This leads directly to the formula below:

Proposition 7.3. *For any edge tuple* $\mathbf{e}$*, the size of its equivalence class under* $\sim$ *is: Let* $U(\mathbf{e}) = \bigcup_{(u,v)\in Set(\mathbf{e})} \{u, v\}$ *be the set of distinct vertices in* $\mathbf{e}$*. Then:*

$$|[\mathbf{e}]_\sim| = \frac{n!}{(n - |U(\mathbf{e})|)!}$$

Proof. Let $\mathbf{e} = ((v_1, v_2), \ldots, (v_{2m-1}, v_{2m}))$ be an edge tuple and let $U = \{v_1, \ldots, v_{2m}\}$ be its vertex set. By definition, $\mathbf{e}' \sim \mathbf{e}$ if and only if there exists a permutation $\pi \in [n]!$ such that

$$\mathbf{e}' = ((\pi(v_1), \pi(v_2)), \ldots, (\pi(v_{2m-1}), \pi(v_{2m})).$$

To count such permutations, we observe that constructing π requires two independent choices. First, we must specify $\pi(u)$ for each $u \in U$, as these values determine the image of $\mathbf{e}$. Second, we can freely assign values to $\pi(u)$ for $u \notin U$, as these do not affect the image of $\mathbf{e}$. Each distinct assignment of values to vertices in U yields a different edge tuple $\mathbf{e}'$.

Therefore, $|[\mathbf{e}]_\sim|$ equals the number of ways to choose $|U|$ distinct values from $[n]$ (for vertices in U) multiplied by $(n - |U|)!$ (for remaining vertices). This yields the claimed formula, as $|U| = |\bigcup_{(u,v)\in Set(\mathbf{e})} \{u, v\}|$. $\square$

8 Representatives of Equivalence Classes

Our next challenge is to compute a representative for each equivalence class of $\sim$ over the set of all ordered m-tuples of edges $([n]^2)^m$. We construct an algorithm GenRepTuples$(c, 2m)$ that generates canonical representatives for these tuples. The key insight is that an ordered tuple of m edges can be viewed as a sequence of $2m$ vertices.

Since relabeling the vertices by any permutation produces an equivalent tuple under $\sim$, we can select a canonical representative for each equivalence class by imposing an ordering constraint on the first appearance of vertex labels. Specifically, as we scan the sequence of $2m$ vertices defining the tuple, we require that each new vertex is introduced in increasing sequential order $(1, 2, 3, \ldots)$. These sequences correspond to Restricted Growth Strings (RGS) of length $2m$. The algorithm GenRepTuples(c, k) generates all such canonical suffixes of length k given that c is the next available new label. The total number of such canonical representatives is given by the Bell number B_{2m}.

The algorithm uses two parameters: c tracks the next available vertex number, while k represents the remaining sequence length. It maintains two invariants: new vertices must be numerically larger than all previous vertices, and when reusing vertices, only values less than c may be used.

The main property of this algorithm is that it generates all possible sequences of length k over the set of vertices while ensuring that the first appearance of each vertex is in increasing order. These sequences correspond to Restricted Growth Strings of length k, which are in bijection with set partitions of a k-element set. As a result, according to Ruskey, the total number of such canonical sequences equals the Bell number $B(k)$ (see [10, Chapter 4.7]).

This property can be understood in terms of an automaton that accepts

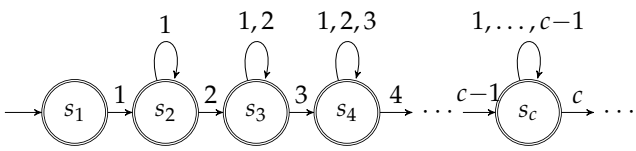
Algorithm 3 GenRepTuples(c, k)

Require: Next available vertex label c, Remaining sequence length k
Ensure: A set of canonical definitions for edge tuples of size compatible with length k

```
if k = 0 then
    return {()}
else
    arr_1 ← {(x, y_1, ..., y_{k-1}) : (y_1, ..., y_{k-1}) ∈ GenRepTuples(c, k−1),
                x ∈ {1, ..., c−1}}
    arr_2 ← {(c, y_1, ..., y_{k-1}) : (y_1, ..., y_{k-1}) ∈ GenRepTuples(c+1, k−1)}
    return arr_1 ∪ arr_2
end if
```

precisely those sequences: the automaton starts in state s_1 and, as it reads each symbol, either stays in the current state (if the symbol corresponds to a previously seen vertex) or transitions to the next state (if a new vertex is introduced in the correct order). The automaton has c states, and the transition function is defined so that a transition from state s_k to s_{k+1} occurs only when the next new vertex $k+1$ appears for the first time. Otherwise, the automaton remains in the current state as long as the symbol corresponds to any of the vertices 1 through k. This construction guarantees that the first occurrence of each vertex is in increasing order, and thus every accepted sequence is a canonical representative for its equivalence class under relabeling.

Proposition 8.1. GenRepTuples$(1, k)$ *produces all the k-tuples in the language of the automaton:*



Proof. We prove by induction on k that GenRepTuples(c, k) generates exactly the set of sequences of length k over $\mathbb{N}$ (starting with the alphabet $\{1, \ldots, c\}$) that satisfy the canonical property: the first occurrence of each number i must be preceded by occurrences of all numbers $1, \ldots, i-1$.

Base case $k = 0$: The only sequence is empty, which is trivially canonical.

Inductive step: Assume GenRepTuples$(c, k-1)$ generates all valid suffixes starting with state c. A canonical sequence of length k starting with established max value $c-1$ can be formed by appending a new symbol x. If $x \in \{1, \ldots, c-1\}$ (handled by arr_1), the set of seen values does not change, and the remaining suffix must be a canonical sequence valid for state c of length $k-1$. If $x = c$ (handled by arr_2), a new value is introduced, so the state updates to $c+1$, and the remaining suffix must be valid for state $c+1$. The algorithm covers exactly these two disjoint cases, thus generating precisely the valid extensions. □

Proposition 8.2. *The set*

$$\{((v_1, v_2), \ldots, (v_{2m-1}, v_{2m})) \colon (v_1, \ldots, v_{2m}) \in \text{GenRepTuples}(1, 2m)\}$$

contains one representative for each equivalence class of $\sim$ *over* $([n]^2)^m$.

Proof. Let $\mathbf{e} = ((v_1, v_2), \ldots, (v_{2m-1}, v_{2m})) \in ([n]^2)^m$ be an arbitrary tuple of m edges. Let $i_1 = 1$ and $i_j = \min\{i > i_{j-1} \colon v_i \notin \{v_1, \ldots, v_{i-1}\}\}$ be the index of the jth new number in the sequence. Define the permutation $\pi(v_{i_j}) = j$ and consider the tuple obtained by applying π to $\mathbf{e}$, denoted as $\mathbf{e}' = ((\pi(v_1), \pi(v_2)), \ldots, (\pi(v_{2m-1}), \pi(v_{2m})))$. By definition, $\mathbf{e}' \sim \mathbf{e}$. To show that $\mathbf{e}' \in$ GenRepTuples$(1, 2m)$, we use Proposition 8.1. Since $\pi(v_i) < j$ for all $i < i_j$ and $\pi(v_{i_j}) = j$ for all j, the automaton can accept the word by transitioning to the next state upon reading the ith letter, for each $i \in I$, and stay in the same state otherwise.

Suppose, in contradiction, that there exist two distinct tuples

$$(v_1, \ldots, v_{2m}), (\pi(v_1), \ldots, \pi(v_{2m})) \in \text{GenRepTuples}(1, 2m)$$

for some $\pi \in [n]!$. We prove, by induction, that π is the identity permutation. The base case $v_1 = \pi(v_1) = 1$ is by definition of GenRepTuples$(1, 2m)$. Inductively, assume that $\pi(i) = i$ for all $i < k$, and we need to prove that $\pi(k) = k$. The assumption means that $\pi(k) \notin \{1, \ldots, k-1\}$ and since $(\pi(v_1), \ldots, \pi(v_{2m})) \in$ GenRepTuples$(1, 2m)$, because all the letters up to k were smaller than k, the automaton must be in state k, and $\pi(k)$ must be k. □

9 Moments of the Omitted Edge Count

We now integrate the components developed in the previous sections into a single algorithmic pipeline for computing the expected number of omitted edges in a random path of length L over the directed graph $([n], [n]^2)$. Given an edge set $S \subseteq [n]^2$, Chapter 5 introduced the automaton $\mathrm{Aut}(n, S)$ that detects whether a path intersects S. Chapter 6 established two efficient methods for counting the number of length L paths that avoid all edges in S, using either direct matrix exponentiation or a Cayley-Hamilton polynomial reduction. Chapter 7 defined an equivalence relation over edge tuples induced by vertex permutations and proved that omission probabilities are identical across each equivalence class. Chapter 8 presented an enumeration algorithm that produces precisely one canonical representative per equivalence class, together with a closed form expression for the size of each class.

Combining these ingredients, our goal is for any fixed m, to evaluate:

$$\mathbb{E}[X^m] = \sum_{\mathbf{e} \in ([n]^2)^m} \Pr[\text{all edges in } \mathbf{e} \text{ are omitted}].$$

Rather than iterating over all $(n^2)^m$ possible ordered tuples, we instead iterate over the much smaller collection of canonical representatives produced by $\mathrm{GenRepTuples}(1, 2m)$. For each representative tuple $\mathbf{e}$, we determine the size of its equivalence class $|[\mathbf{e}]_\sim|$ using Proposition 4, construct the automaton $\mathrm{Aut}(n, Set(\mathbf{e}))$ for the underlying set of edges, compute the number of accepted paths of length L, and multiply this count by the class size. Summing these values over all representatives and normalizing by the total number of paths n^{L+1} yields the m-th moment of the omitted edge count.

The following algorithm summarizes this computation.

Algorithm 4 OmissionMoment(n, L, m)

Require: Alphabet size n, Path length L, Moment order m
Ensure: The m-th moment of the number of omitted edges $\mathbb{E}[X^m]$

```
R ← GenRepTuples(1, 2m)          {Generate canonical representative tuples}
Total ← 0
for each representative tuple e ∈ R do
    C ← |[e]_~|                  {Equivalence class size (Proposition 4)}
    N ← PathCount(n, L, Set(e))  {Using Algorithm 1 or Algorithm 2 on induced edge set}
    Total ← Total + C · N
end for
moment ← Total/n^(L+1)
return moment                    {Returns E[X^m]}
```

10 Evaluation: The OPSO Case Study

The OPSO (Overlapping Pairs Sparse Occupancy) test, introduced decades ago by George Marsaglia, assesses the quality of pseudorandom number generators by examining the frequency of overlapping two-letter words in a stream of random letters. It is widely used as part of the Diehard battery of tests [7].

The statistical question behind the OPSO test can be abstracted as a problem in combinatorics: given a finite alphabet Σ and a stream of length $L+1$ over Σ, how many possible overlapping words of length n are missing from the stream? Understanding the distribution of this count, especially its expectation, variance, and higher moments, is crucial for calibrating the test and evaluating its sensitivity. The actual numbers are $L = 2^{21} = 2{,}097{,}152$ and $n = 2^{10} = 1,024$.

This problem was first posed by Marsaglia [7], and later addressed by Leopardi [6], who used generating function techniques to derive formulas for the expectation and variance. His work, based on correlation polynomials, achieves 20-digit precision in these two moments but does not extend easily to higher moments or provide runtime measurements (see Section 10.2).

Rahmann and Rivals [8] explicitly note that their generating function approach does not extend to higher moments and leave their exact computation open; in this work, we resolve this problem by modeling the word omission process as a finite automaton and applying algebraic techniques. We compute all moments of the distribution, including skewness and kurtosis, with full precision. This not only recovers the known results but also extends them, offering insights into the tail behavior and convergence properties of the distribution.

Our method is efficient and scalable, enabling the analysis of large values

of n and L that were previously impossible. The approach provides a new tool for understanding subtle statistical properties in pseudorandom sequences and has applications in the design and evaluation of randomness tests.

The OPSO test specifically asks for the distribution of the number of missing 2-grams ($m = 1$) or higher-order configurations. As detailed in Chapter 4, this corresponds to the moments of the variable X. For the OPSO test in the Diehard suite, the parameters are $n = 1024$ and $L = 2^{21}$.

10.1 Numerical Results

We evaluate the computational efficiency and numerical accuracy of our methods for computing omission probabilities and moments in the OPSO test. All experiments target the standard parameters where $n = 1024$ and $L = 2^{21}$. We emphasize scaling with respect to the number of forbidden edges m, and compare two algorithmic approaches: direct matrix exponentiation and a polynomial method based on the Cayley-Hamilton theorem.

10.1.1 Computational Performance

Table 10.1 and Figure 10.1 report the runtime and speedup of the polynomial method over direct matrix power as m increases. The number of equivalence classes and unique automata grows rapidly with m, but remains tractable due to structural reductions.

Observations. The polynomial method consistently outperforms the matrix-based approach. For example, when $m = 4$, the runtime drops from 2584 seconds using matrix exponentiation to 287 seconds with the polynomial method, achieving a $9\times$ speedup. This improvement stems from avoiding explicit matrix powers and instead evaluating low degree matrix polynomials, significantly reducing the computational burden.

For the specific case of the first two moments ($m = 1, 2$), our implementation of the fully analytical generating function method provides the fastest results: 0.18 seconds for the expectation and 1.60 seconds for the second moment (1.78 seconds total). This confirms that for standard parameters requiring only variance, the analytical approach is orders of magnitude more efficient than the general-purpose matrix methods required for

m	**Classes**	**Automata**	**Matrix (s)**	**Poly. (s)**	**Gen. Func. (s)**	**Speedup (×)**
1	2	2	1.46	1.13	0.18	1.29
2	15	14	5.37	2.72	1.60	1.97
3	203	126	76.63	19.07	–	4.02
4	4140	1518	2584.17	287.19	–	9.00
5	115,975	22,941	–	5431.10	–	–

Table 10.1: Runtime comparison between matrix exponentiation, the polynomial method, and the analytical generating function method (for $m \leq 2$). **Equiv.**: number of equivalence classes of edge sets. **Automata**: number of unique automata constructed. **Matrix (s)**: total runtime using matrix exponentiation (in seconds). **Poly. (s)**: total runtime using the polynomial method (in seconds). **Gen. Func. (s)**: total runtime using the analytical generating function method (in seconds), an implementation utilized by Leopardi [6] based on the work of Rahmann and Rivals [8]. **Speedup (×)**: speedup of Polynomial over Matrix.

higher moments (e.g., Matrix Power takes 2584s for $m = 4$). However, for analyzing higher-order statistical properties ($m \geq 3$), our automata-based polynomial method remains the most effective tool.

10.1.2 Accuracy and Moment Estimation

Table 10.2 presents exact omission probabilities and the first four raw moments of the number of missing edge pairs. These match known empirical OPSO test statistics and demonstrate the effectiveness of our high precision implementation.

m	**Omission Prob.**	**Raw Moment** $E[X^m]$
1	0.1353352832	141909.329955006918907401081611 1...
2	0.0183157156	20138342296.60548564754247221057...
3	0.0024787833	2857842607047560.855221564201407...
4	0.0003354711	405559626608917536786.6404627461...
5	0.0000454018	57553659317649852599949683.03906...

Table 10.2: Computed omission probabilities and raw moments for increasing m. Values computed using the polynomial method.

Observations. The omission probability decays exponentially with increasing m, while the raw moments grow rapidly demonstrating the combinatorial explosion of omission configurations. Exact moment values are essential for distinguishing true randomness from subtle biases in random number generators.

10.2 Comparison with Related Methods

Prior work by Leopardi [6] successfully addressed the statistics of the OPSO test using generating function techniques. This approach derives the expectation and variance with high numerical precision; indeed, the expected values reported by Leopardi match the first 20 digits of our independently derived results. However, while Leopardi focuses exclusively on the first two moments, our framework extends naturally to computing higher moments, such as skewness and kurtosis. These are critical for analyzing tail behavior and improving the accuracy of estimates.

The limitation in previous work is not merely computational but theoretical. Extending the generating function computations used by Leopardi to higher-order moments requires characterizing and enumerating $m \times m$ correlation matrices of pairwise overlaps between m distinct words. Rahmann and Rivals explicitly identified this combinatorial explosion as an open problem [8], noting that the structure of these matrices is not fully understood.

Our automata-based framework bypasses this theoretical bottleneck entirely. Instead of characterizing correlation matrices analytically, we solve the counting problem algorithmically via the transition matrix of the automaton $\mathrm{Aut}(n, S(e))$.

We implemented the analytical generating function method for comparison, and as shown in Table 10.1 and Figure 10.1, generating functions are faster for the first two moments ($m = 1, 2$), computing exact values in under two seconds. However, this approach does not easily extend to higher moments ($m \geq 3$). In contrast, our automata-based approach provides the first general purpose solution for arbitrary moments, enabling both exact results and controlled approximations with significantly reduced computational overhead for $m \geq 3$.

Our automata-based approach also enjoys strong inherent numerical stability. The problem is formulated as counting paths in a finite graph, so all intermediate computations, whether matrix multiplications or polyno-

mial arithmetic, are carried out exactly over the integers. As a result, the method produces an exact integer count of valid paths, entirely free of numerical approximation or precision loss.

By contrast, generating-function-based approaches typically rely on extracting and manipulating polynomial roots, which are often irrational or complex, and summing their powers. These operations are usually implemented using floating-point arithmetic and are therefore susceptible to numerical instability. In our experiments, this instability manifested in spurious artifacts, such as tiny imaginary components ($1.75 \times 10^{-46} i$) appearing in quantities that are real-valued probabilities.

Our approach avoids such issues by construction: all computations remain integer-exact until the final normalization step. The only rounding occurs in the last step of Algorithm 4, where the exact path count is divided by the total number of paths to obtain a probability. Consequently, the numerical accuracy of the final result is controllable and can be made arbitrarily precise.

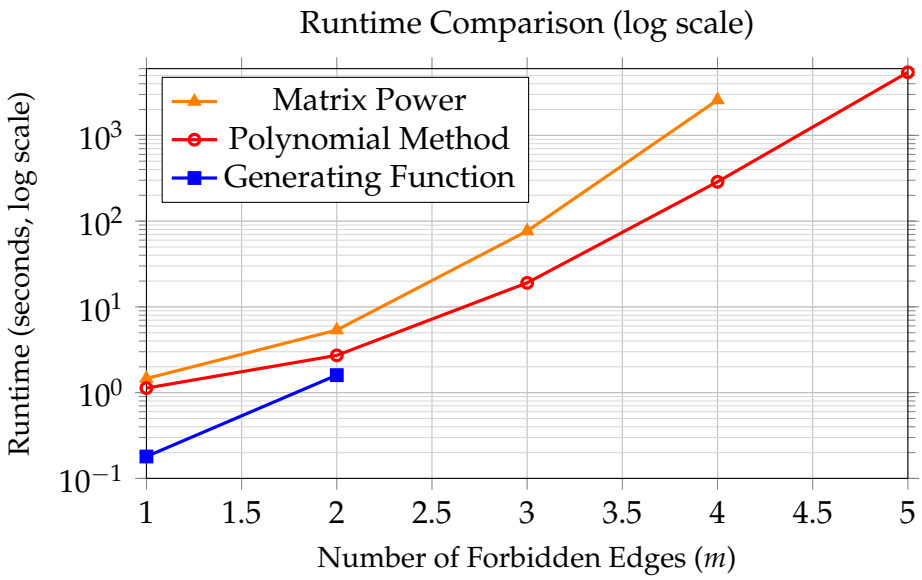


Figure 10.1: Runtime comparison (log scale) between three methods for computing omission probabilities. The x-axis shows the number of forbidden edges m, and the y-axis shows runtime in seconds (logarithmic scale). Each point corresponds to the total runtime for processing all equivalence classes at a given m. The **Generating Function** line (blue) represents the fully analytical method based on correlation polynomials, as utilized by Leopardi [6] following Rahmann and Rivals [8]. This method computes the first two moments with the fastest runtime (0.18s for m=1, 1.60s for m=2), but has no data for $m \geq 3$ because the underlying theory does not extend to higher moments. The **Matrix Power** method (orange) uses direct matrix exponentiation. The **Polynomial Method** (red) achieves substantial speedups by replacing repeated matrix multiplications with evaluations of low degree matrix polynomials via the Cayley-Hamilton theorem, and is essential for computing moments with $m \geq 3$.

11 Discussion

In this chapter, we synthesize the findings from our automata-theoretic framework and the numerical results obtained for the OPSO test. We analyze the scalability of the approach and discuss the statistical implications of having access to exact higher-order moments. Finally, we outline future research directions and potential applications in bioinformatics and natural language processing.

11.1 Positioning Relative to Related Work

This thesis builds upon and extends the progression of work by Rahmann, Rivals, Leopardi, and Azran discussed in Chapter 3. Rahmann and Rivals established the generating function framework and asymptotic behavior of missing q-grams. Leopardi demonstrated how these methods can be pushed to obtain extremely accurate numerical values for low-order moments and highlighted the practical importance of exact theoretical parameters in empirical testing. Azran introduced an automata- and matrix-based formulation that enabled exact, non-asymptotic computation of the expectation and variance of the OPSO statistic.

This thesis synthesizes and extends these approaches. Building on Azran's automata-based model, we develop a complete algorithmic framework for computing arbitrary higher-order moments of missing word statistics. By introducing a canonical enumeration of equivalence classes via restricted growth strings, analyzing the resulting complexity in terms of Bell numbers, and employing Cayley-Hamilton-based matrix reduction, we transform formal moment expansions into a fully operational computational pipeline.

In doing so, this thesis reproduces and validates the high precision results of Leopardi for the first two moments, aligns with the theoretical insights of Rahmann and Rivals, and goes beyond prior work by enabling exact

computation of higher moments. This positions the thesis as a unifying and extending contribution to the theory and practice of randomness testing based on missing pattern statistics.

11.2 Interpretation of the Experimental Results

The numerical results presented in Section 10.1 reveal key characteristics of the distribution of missing 2-grams. First, the probability of omitting specific sets of edges decays exponentially with the size of the set, which is expected. However, the exact rate of this decay is governed by the overlap structure of the counting automaton. Our method successfully captures these fine-grained structural dependencies. For example, a set of missing edges that forms a path is less probable than a set of disjoint edges, because the path constraint imposes a stricter correlation on the sequence. The ability to compute these differences exactly allows for a much more rigorous calibration of randomness tests.

11.3 Strengths of the Automata-Theoretic Approach

The primary strength of our framework is its *exactness*. Unlike asymptotic approximations which hold only when $L \to \infty$ or $n \to \infty$, our method provides precise integer counts for any finite L and n. This is achieved by transforming the combinatorial counting problem into a linear algebraic one. Furthermore, the use of equivalence classes allows us to compress the state space significantly. As shown in Table 10.1, the number of unique automata is orders of magnitude smaller than the total number of edge tuples. This structural insight turns a computationally intractable problem into a feasible one for moderate m.

11.4 Scalability Limits and Bell Numbers

While our approach is highly efficient with respect to the sequence length L (logarithmic complexity), it faces a combinatorial bottleneck with respect to the moment order m. The number of equivalence classes is related to the Bell number B_{2m}, which grows superexponentially. For $m = 1$, we have 2 classes. For $m = 4$, we have 4140. For $m = 5$, the number jumps to over 115,000, and for $m = 6$, it would exceed 6 million. This indicates that while our method is excellent for the first few moments (sufficient for

skewness and kurtosis), computing extremely high-order moments (e.g., $m = 10$) remains computationally expensive. However, for statistical testing purposes, the first four moments are typically sufficient to characterize the distribution's shape and tail behavior accurately.

11.5 Statistical Meaning of Higher Moments

The computation of skewness (3rd moment) and kurtosis (4th moment) provides deeper insight into the null distribution of the OPSO statistic. A non-zero skewness indicates that the distribution is not symmetric around its mean; knowing this allows us to adjust confidence intervals to be asymmetric, improving the test's power. High kurtosis indicates "fat tails," meaning that extreme deviations from the mean are more likely than in a Normal distribution. Failure to account for kurtosis can lead to false positives in randomness testing, rejecting a good generator because a rare event occurred that was actually consistent with the true (leptokurtic) distribution. Our exact moments thus provide the necessary constants for Edgeworth expansions or other higher-order refinements of the Central Limit Theorem.

11.6 Future Directions and Applications

11.6.1 Extensions of the Framework

Generalizing to k-grams. The current work focuses on 2-grams (pairs). The OQSO test involves 4-grams. Extending our framework to k-grams for $k > 2$ is conceptually straightforward but computationally demanding. An edge would become a hyperedge or a path of length $k - 1$. The state space of the automaton would enable transitions based on the $(k-1)$-suffix. The combinatorial challenge is that the size of the equivalence classes (and the number of such classes) grows much faster. Future work could investigate efficient generating algorithms for these higher-order equivalence classes, potentially utilizing de Bruijn graph automorphisms.

11.6.2 Other Possible Applications of the Framework

The study of minimal absent words, or "nullomers," is a significant area of research in bioinformatics. Acquisti et al. [1] investigated the origins of

these sequences, debating whether they arise from negative selection due to lethality or from mutational biases such as CpG hypermutability. Our framework may contribute to this inquiry by providing a precise method to compute the expected frequency and higher-order moments of missing patterns under neutral random models, thus offering a rigorous baseline for distinguishing selective pressure from background statistical noise.

In natural language processing, language models frequently encounter *data sparsity*, where many valid n-grams do not occur in the training corpus. Chen and Goodman [3] provided an extensive empirical study of smoothing techniques designed to address this challenge by discounting observed counts to assign non-zero probabilities to unseen events. Our work may offer a complementary perspective by providing exact theoretical moments for the omission count distribution under a random source model, which can serve as a baseline for evaluating smoothing algorithms.

While we focused on the OPSO test as a benchmark, many other statistical tests for randomness rely on pattern occurrences and could benefit from our framework. For instance, the NIST statistical test suite described by Rukhin et al. [9] includes several tests based on pattern frequencies. Our ability to provide exact moments and tail behavior for omission distributions may offer a path toward refining these protocols and improving the sensitivity of randomness certification.

11.7 Conclusion

This thesis closes the problem of computing exact higher moments for the number of missing 2-grams. By essentially "mechanizing" the resolution of overlaps through automata and matrix algebra, we have bypassed the combinatorial barrier that stalled previous generating function approaches. The necessity of classifying high-order correlation matrices is rendered obsolete; instead, the problem is reduced to the systematic enumeration of symmetry classes of automata states.

While the extension to general k-grams (such as for the OQSO test) remains computationally demanding due to state space explosion, the theoretical path is clear and no longer blocked by missing conceptual tools. For the domain of randomness testing, this work marks a shift from asymptotic heuristics to exact certification. With the ability to compute arbitrary moments, we can now characterize the exact shape of the null distribution, including its skewness and kurtosis, allowing for precise p-values even in the rigorous tails of the distribution. This capability transforms the OPSO

test from a "black box" indicator into a precision instrument for validating the fundamental properties of random sources.

A Implementation Notes

The complete source code for this thesis is available at `https://github.com/KiralyOr/opso`. This repository contains the full reference implementation of the algorithms described in Chapter 9, bridging the theoretical framework with practical realization.

A.1 Software Structure and Implementation

The codebase is organized into two primary components: the automata-based framework and a baseline generating function implementation. The core logic resides in the `opso/` directory, which implements the exact automata-based method using Cayley-Hamilton reduction. This module features caching optimizations to avoid redundant computations and leverages parallel task execution via `concurrent.futures` to distribute equivalence class processing across available CPU cores. Complementing this, the `opso/generating_function_implementation/` directory provides a reference implementation of the classical generating function approach, serving as a verification baseline for low-order moments.

Implementation Details. All moment computations were implemented using `sympy` (v1.13.3) for symbolic matrix and polynomial manipulation. To handle the large integer and rational coefficients that arise in higher-order moments, we utilized the `gmpy2` library (v2.2.1) as a high-performance backend. By delegating multi-precision arithmetic to the `GMP` and `MPFR` C-libraries, we achieved significant performance gains over native Python integers. This optimization was essential for processing the large state-space and high-degree polynomials required for $m \geq 4$.

A.2 Reproducibility

To reproduce the results, the provided code requires a Python 3.8+ environment with the dependencies `sympy`, `gmpy2`, and `tqdm` installed. The main pipeline can be executed via:

```
python3 opso/batch_calculate_total_sum_\
        with_log_poly.py
```

This script runs the full computation for moments $m = 1$ to $m = 4$, logging runtime metrics and saving exact results to `.pkl` files. The generating function baseline can be run separately for verification of the first two moments.

Bibliography

[1] Claudia Acquisti, George Poste, David Curtiss, and Sudhir Kumar. Nullomers: Really a matter of natural selection? *PLOS ONE*, 2(10):1–3, 10 2007. doi:10.1371/journal.pone.0001022.

[2] Yahav Azran. Generalizing the opso pseudo random number generator test and its analysis. Master's thesis, Ben-Gurion University of the Negev, 2022.

[3] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–394, 1999. doi:10.1006/csla.1999.0128.

[4] Leonidas J. Guibas and Andrew M. Odlyzko. String overlaps, pattern matching, and nontransitive games. *Journal of Combinatorial Theory, Series A*, 30(2):183–208, 1981. doi:10.1016/0097-3165(81)90005-4.

[5] David Harvey and Joris van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021. doi:10.4007/annals.2021.193.2.4.

[6] Paul Leopardi. Testing the tests: Using random number generators to improve empirical tests. In Pierre L'Ecuyer and Art B. Owen, editors, *Monte Carlo and Quasi-Monte Carlo Methods 2008*, pages 501–512, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. URL: https://maths-people.anu.edu.au/~leopardi/Leopardi-TestU01-paper-final.pdf.

[7] George Marsaglia. Diehard: A battery of tests of randomness, 1996. URL: http://stat.fsu.edu/pub/diehard/.

[8] Sven Rahmann and Eric Rivals. On the distribution of the number of missing words in random texts. *Combinatorics, Probability*

and Computing, 12(1):73–87, 2003. URL: https://www.lirmm.fr/~rivals/PUBLI/FILES/Rahmann-Rivals-CPC-03.pdf, doi:10.1017/S0963548302005473.

[9] Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, Stefan Leigh, Mark Levenson, Mark Vangel, David Banks, and Alan Heckert. A statistical test suite for random and pseudo-random number generators for cryptographic applications. Technical Report Special Publication 800-22 Revision 1a, National Institute of Standards and Technology, 2010. doi:10.6028/NIST.SP.800-22r1a.

[10] Frank Ruskey. Combinatorial generation, 1995. URL: https://page.math.tu-berlin.de/%7Efelsner/SemWS17-18/Ruskey-Comb-Gen.pdf.

[11] Richard P. Stanley. *Enumerative Combinatorics*, volume 1. Cambridge University Press, 2011. URL: http://math.mit.edu/~rstan/ec/.